



—
Dfns

SMART CONTRACT AUDIT REPORT

30 MAY 2025

Audited by: RAID Square

RDS-CS-0042-B

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025



Table of Contents

- 1. Introduction**
 - 1.1. About RAID Square
 - 1.2. About Dfns
 - 1.3. Mission
 - 1.4. Methodology
 - 1.5. Disclaimer

- 2. Findings**
 - 2.1. Summary

- 3. Detailed Results**

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025



Revision History

Revision	Date	Amendment
A	30/05/2025	Initial version.
B	19/06/2025	Final version.

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

1. INTRODUCTION

As part of a request to improve the security of their protocol, Dfns commissioned RAID Square to carry out an audit of the various smart contracts listed below in the "Mission" section. This document highlights the results of the audit that was carried out and highlights the risks and opportunities for improvement from the current state of the contracts at the time of delivery.

1.1 About RAID Square

RAID Square is a French company specialised in risk mitigation for Blockchain stakeholders. RAID Square's offerings are structured around the following four business verticals:

- Cybersecurity & audits cross WEB2 X WEB3 ;
- On-chain investigations (+OSINT & HUMINT) & Due diligence ;
- Legal structuring and coordination of WEB3 legal procedures ;

1.1 About Dfns

Dfns is the WaaS (wallets-as-a-service) platform chosen by leading fintechs and enterprises to streamline digital asset operations and onboard users into crypto, safely.

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision:	Doc. Type:	Language:	Total Pages:	Owner:	Status:	Issued on :
B	PR	EN	19	RAID Square	VA	30 May 2025

1. 3 Mission

Item	Description
Project name	Dfns
Website	https://www.dfns.co/
Type	Smart Contract
Language	Solidity
Mission start date	23 May. 2025
Mission end date	30 May. 2025
Version	1.0.0

The RAID Square mission is to carry out audits of the contracts listed below:

- Smart Contract #1: **DfnsSmartAccount.sol**
 - <https://github.com/dfns/dfns-smart-account/blob/m/src/DfnsSmartAccount.sol>

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

1. 3 Mission

Smart Contract	Hash sha256
A: DfnsSmartAccount.sol	<i>3154fbec1c91d0c15532fc956f019fbb0c98738d99ec0a3bf993deafe8c4ae27</i>
B: DfnsSmartAccount.sol	<i>05016c490face49ffc900e52c7eb061d645ae4c3747cb01d035d0e1d83cdc09b</i>

The calculation of the SHA-256 hash of each smart contract as part of this audit may allow verification of the version of the code that was audited by the RAID Square team.

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

1. 4 Methodology

A smart contract audit is a process of examining and assessing the security and quality of a smart contract, with the aim of identifying vulnerabilities and ensuring that the contract is working as intended. There are several different methodologies that can be used to conduct a smart contract audit, each with its own set of best practices and considerations.

It is important to note that a smart contract audit is not a one-off process, but rather an ongoing effort to ensure that the contract remains secure and functions as intended. Regular audits should be conducted to identify and resolve any issues that may arise over time.

RAID Square has created its audit methodology based on different existing models and bases its approach on the notion of risk.

Risk = Likelihood * Impact

Risk Severity				
Impact	High	Medium	High	Critical
	Medium	Intermediary	Medium	High
	Low	Low	Intermediary	Medium
		High	Medium	Low
	Likelihood			

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision:	Doc. Type:	Language:	Total Pages:	Owner:	Status:	Issued on :
B	PR	EN	19	RAID Square	VA	30 May 2025

1. 4 Methodology

The probability estimate and the impact estimate are put together to calculate an overall severity for each risk. This is done by determining whether the probability is low, intermediary, medium, high or c and then doing the same for the impact. The 0 to 10 scale:

Likelihood and Impact Levels	
0 to <2	Low
2 to <4	Intermediary
4 to <6	Medium
6 to <9	High
10	Critical

In order to determine the probability, we use the following guiding factors for each vulnerability found:

Skill Level - The level of skill required to carry out the attack:

- ☐ No technical skills (1 to 3)
- ☐ Some technical skills (3 to 5)
- ☐ Advanced computer user (5 to 6)
- ☐ Networking and programming skills (6 to 9)
- ☐ Security penetration skills (9 to 10)

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

1. 4 Methodology

Motivation - How motivated is the attacker to find and exploit the vulnerability?

- ☐ No reward (1 to 2)
- ☐ Reward less than the cost of the attack (2 to 4)
- ☐ Reward at break-even cost of attack (4 to 6)
- ☐ Reward greater than the cost of the attack (6 to 9)
- ☐ High or full reward (10)

Opportunity - What resources are required for the attacker to find and exploit this vulnerability?

- ☐ Full access or expensive resources required (0 to 2)
- ☐ Access or special resources required (2 to 7)
- ☐ Some access or few resources required (7 to 9)
- ☐ No access or resources required (9 to 10)

Status - What is the status of the attacker?

- ☐ Developers (2)
- ☐ System administrators (2 to 4)
- ☐ Intranet users (4 to 5)
- ☐ Partners (5 to 6)
- ☐ Authenticated users (6 to 9)
- ☐ Anonymous Users (9 to 10)

Discovery difficulty - How easy is it for the attacker to discover this vulnerability?

- ☐ Nearly impossible (1 to 2)
- ☐ Difficult (3 to 7)
- ☐ Easy (8 to 9)
- ☐ Automated tools available (10)

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

1. 4 Methodology

Exploitation difficulty - How easy is it for the attacker to actually exploit this vulnerability?

- ☐ Theoretical (1 to 2)
- ☐ Difficult (3 to 4)
- ☐ Easy (5 to 9)
- ☐ Automated tools available (10)

Awareness - To what extent is this vulnerability known to the attacker?

- ☐ Unknown (1 to 3)
- ☐ Hidden (4 to 5)
- ☐ Obvious (6 to 9)
- ☐ Public knowledge (10)

Detection - How likely is this exploit to be detected?

- ☐ Active detection in the application (1 to 2)
- ☐ Logged and reviewed (3 to 7)
- ☐ Logged without review (8 to 9)
- ☐ Not logged (10)

In order to determine the impact, we use the following guiding factors for each vulnerability found:

Confidentiality - How much data could be disclosed and how sensitive is it (if linked to a central database)?

- ☐ Minimum non-sensitive data disclosed (2 to 5)
- ☐ Minimum critical data disclosed (6 to 7)
- ☐ Extended non-sensitive data disclosed (6 to 7)
- ☐ Extended critical data disclosed (7 to 9)
- ☐ All data disclosed (10)

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

1. 4 Methodology

Integrity - How much data could be corrupted and how badly is it damaged?

- ☐ Minimal data slightly corrupted (1 to 2)
- ☐ Minimal data severely corrupted (3 to 4)
- ☐ Extensive data slightly corrupted (5 to 6)
- ☐ Severely corrupted extended data (7 to 9)
- ☐ All data totally corrupted (10)

Non-compliance - What degree of exposure does non-compliance cause?

- ☐ Minor violation (2 to 4)
- ☐ Open violation (5 to 6)
- ☐ High-profile violation (7 to 10)

Availability - How much service could be lost and how vital is it?

- ☐ Minimal secondary services interrupted (1 to 4)
- ☐ Minimal primary services interrupted (5 to 6)
- ☐ Extended secondary services interrupted (5 to 6)
- ☐ Extended primary services interrupted (7 to 9)
- ☐ All services totally lost (10)

Finance - What will be the financial damage from an exploit?

- ☐ Less than the cost of fixing the vulnerability (1 to 2)
- ☐ Minor effect on annual profit (3 to 6)
- ☐ Major effect on annual profit (7 to 9)
- ☐ Bankruptcy (10)

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

1. 4 Disclaimer

RAID Square is performing a technical audit of the smart contract code as an independent service provider. We have performed this audit in a diligent and professional manner, but it is important to understand that our job is only to verify the quality and security of the code in its current state.

This security audit is not intended to replace the functional testing required before any software release. Furthermore, it does not guarantee the discovery of all possible security issues of the given smart contract(s) or blockchain software. In other words, the result of the evaluation does not guarantee the non-existence of any other security issues.

As an single audit-based evaluation cannot be considered complete, we always recommend several independent audits and a public bug bounty programs to ensure the security of the smart contract(s).

We are not responsible for any security flaws that may be discovered later, nor for any damage that may result from them. Similarly, we are not responsible for any changes made to the code after the audit has been completed.

It is the responsibility of users and stakeholders to exercise due diligence when using the audited smart contracts and to regularly check their security. RAID Square accepts no liability for any loss or damage arising from the use of or reliance on these smart contracts.

Last but not least, this security audit should not be used as investment advice.

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

2. FINDINGS

2.1 Summary

Here is a summary of our findings after analysing the implementation of the Dfns Smart Contract. During the first phase of our audit, we studied the source code of the smart contracts using our internal static code analyser. After identifying known coding bugs and checking them manually, we manually examined the logic of the code for potential bugs.

Severity	Number of findings
Low	5
Intermediary	0
Medium	1
High	0
Critical	0
Total	6

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

3. DETAILED RESULTS

3.1 DfnsSmartAccount.sol

3.1.1 Floating Compiler Version — Low — FIXED

```
pragma solidity ^0.8.29;
```

Description: Locking the pragma ensures that contracts are not accidentally deployed using, for example, an obsolete compiler version that could introduce bugs that adversely affect the contract system.

Recommendation: Use fixed solidity compiler version.

Code:

```
pragma solidity =0.8.29;
```

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025



3.1.2 Incomplete Documentation – Low – TO FIX

Description: While the README file includes instructions for deploying the contract, verifying it, and running tests, it lacks essential guidance for developers or integrators looking to use the contract in a real environment. Specifically, it does not provide clear instructions on how to integrate the contract with an EOA (Externally Owned Account) or how to construct and send `UserOperation` (userOp) objects. Additionally, there are a few inconsistencies in the documentation: the deployment command references the wrong script (`DfnsSmartAccount` instead of `script/DeployDfnsSmartContract.s.sol`), and the expected structure and meaning of the `UserOp` parameters, particularly for functions like `handleOps`, are not properly documented.

Recommendation: Revise the README to provide more complete integration instructions. In addition to deployment, verification, and test commands, the documentation should include a clear explanation of how an external user (e.g., an EOA) can interact with the contract, including how to construct valid `UserOperation` objects. Correct any inaccurate references to scripts, such as replacing `DfnsSmartAccount` with the correct path `script/DeployDfnsSmartContract.s.sol`, and clearly define the purpose and expected format of parameters in key functions like `handleOps`. Including a practical usage example and data structure references would significantly improve the readability and usability of the documentation.

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

3.1.3 Unsafe Deployment Practices and Unused Variable Declaration — Low — FIXED

Description: The current deployment guidance recommends using the `--private-key` flag with the `forge create` command. This practice is highly discouraged, as it can lead to unintentional exposure of sensitive credentials, particularly for less experienced developers. Additionally, the documentation does not suggest more secure alternatives, such as the use of hardware wallets through the `--ledger` option. Moreover, the smart contract includes an `immutable` and `private` variable named `ENTRY_POINT`, inspired by `safeLite.sol`. However, this variable is not used in any meaningful way, and its presence may cause confusion or suggest an unimplemented feature.

Recommendation: Avoid the use of `--private-key` in documentation and example scripts. Instead, promote safer deployment options like `--ledger`, which allows the use of hardware wallets. Review the necessity of the `ENTRY_POINT` variable. If it serves no practical purpose in the current contract design, it should be removed to maintain clean and understandable code.

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025

3.1.4 Inefficient Bitwise Operations and Lack of Low-Level Optimization in Signature Validation – Low – FIXED

Description: The `validateSignature` function includes the use of the `shr()` opcode to perform right-shift operations. While functionally correct, using the native Solidity bitwise operator `>>` is more gas-efficient due to compiler-level optimizations. Beyond this, the function as a whole could benefit from deeper gas optimization, particularly considering that signature validation may be executed frequently and can become a bottleneck in high-throughput scenarios. Solidity's high-level implementation, although readable, may introduce unnecessary overhead compared to an inline assembly implementation tailored for ECDSA parsing and address recovery.

Recommendation: Replace the usage of `shr()` with the native `>>` operator in the Solidity code to take advantage of compiler-level optimizations. For more advanced gas savings, consider rewriting the `isValidSignature` function in inline `assembly`. A carefully designed low-level version can optimize stack usage, remove redundant checks, and streamline memory access, resulting in significantly lower gas costs during signature validation. Ensure that any such low-level implementation remains secure, well-documented, and thoroughly tested to prevent regressions or vulnerabilities.

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision: B	Doc. Type: PR	Language: EN	Total Pages: 19	Owner: RAID Square	Status: VA	Issued on : 30 May 2025



3.1.5 Lack of Bounds and Target Address Validation in UserOperation Processing — Low — FIXED

Description: During unit and fuzzing tests, it was discovered that the contract fails to robustly validate the `bytes` data of incoming `UserOperation` (UserOp) structures. Specifically, malformed or boundary-edge UserOps with crafted combinations of `value` and `data` can trigger undefined behavior. For instance, in the test case `test_Assembly_ValueExtraction_EdgeCases`, unexpected reverts occurred due to invalid memory access or calls made to the zero address. One class of issues was related to missing validation of the target address: certain test cases attempted balance checks or operations involving `address(0)`, which should be considered invalid. Additionally, when `dataLength` was incorrectly calculated or exceeded expected bounds, the contract attempted out-of-bounds memory reads, potentially reverting or leading to incorrect state changes.

Recommendation: To mitigate the risk of undefined behavior during the parsing and execution of `UserOperation` data, implement strict validation checks directly within the assembly block. First, ensure that the extracted target address is not the zero address (`address(0)`); if it is, the contract should revert with a clear and explicit error such as `InvalidTarget()`, rather than allowing the call to proceed or fail silently. Second, validate that any memory access involving the `dataLength` does not exceed the total length of the `UserOperation`. This can be enforced by checking that the calculated offset specifically, the sum of the iterator index, base offset (typically `0x54`), and `dataLength` is less than or equal to the total size of the `UserOperation` bytes. These two checks help prevent both invalid external calls and out-of-bounds memory access, improving the reliability and security of the contract's execution path under edge-case conditions.

DISTRIBUTION: CONTROLLED

			Smart Contract Audit Dfns			Document No.: RDS-CS-0042
Revision:	Doc. Type:	Language:	Total Pages:	Owner:	Status:	Issued on :
B	PR	EN	19	RAID Square	VA	30 May 2025

3.1.6 Incomplete Signature Malleability Protections in

`isValidSignature` – Medium – FIXED

Description: The `isValidSignature` function (lines 76–84) manually unpacks and validates an ECDSA signature by extracting the `v` and `s` values and calling the `ecrecover` precompile. However, this implementation lacks a critical constraint: it does not enforce the EIP-2 requirement that the `s` value must be in the lower half of the secp256k1 curve order ($s \leq N/2$). While EIP-2 applies this rule at the transaction level, it does not apply to contract-level calls to `ecrecover`, meaning this function is susceptible to signature malleability attacks. This is particularly problematic when handling signatures conforming to EIP-712 or EIP-2098, which may still be accepted in malleable form if no additional check is performed. Additionally, since the logic is placed inside an `unchecked` block, failures may result in a silent `address(0)` comparison rather than a clear revert, which reduces code safety and debuggability.

Recommendation: To protect against signature malleability, implement stricter validation on the `s` value. For a gas-unoptimized but more secure approach, consider using OpenZeppelin's `ECDSA.recover` function (from version 4.7.0 or above), which includes proper checks on `s` and produces explicit revert errors in case of invalid signatures. For a more gas-efficient solution, perform a manual check to ensure that $s \leq \text{CURVE_ORDER} / 2$, and then use inline assembly to call the `ecrecover` precompile. This can be done by preparing the signature data in memory, using a `staticcall` to `0x01`, and comparing the recovered address to the expected signer. Additionally, include clear revert messages (e.g., `InvalidSignatureSValue`, `SignatureRecoveryFailed`) to aid debugging and ensure that invalid cases do not silently fail. These changes will eliminate malleability risks and improve both security and clarity of the contract's signature validation process.

DISTRIBUTION: CONTROLLED